

04. AI - Học Và Hiểu Library74 Từ Gốc

Tài liệu này xem bạn là người mới học lập trình web ở mức cơ bản. Mục tiêu không phải học thuộc code, mà là hiểu phần AI đang giải quyết vấn đề gì, vì sao chọn cách đó, và khi hội đồng hỏi thì bạn giải thích được bằng logic.

Lộ trình học toàn project:

1. FE - Frontend: người dùng nhìn thấy gì, bấm gì, dữ liệu đi đâu.
2. DB - Database: dữ liệu được lưu như thế nào.
3. BE - Backend: nghiệp vụ, bảo mật, API.
4. AI - Recommendation, semantic search, xử lý tri thức.
5. Deploy/DevOps - đưa hệ thống chạy thật trên server.

Trong file này ta học phần AI.

1. AI Trong Library74 Giải Quyết Vấn Đề Gì?

1.1. Vấn Đề Thực Tế

Một hệ thống thư viện truyền thống thường tìm sách theo:

- Tên sách.
- Tác giả.
- ISBN.
- Danh mục.
- Tag thủ công.

Cách này tốt nhưng chưa đủ. Người dùng có thể tìm bằng ý tưởng, không nhớ đúng tên sách.

Ví dụ người dùng nhập:

sách giúp tôi học tư duy thiết kế hệ thống backend

Nếu database chỉ tìm chữ khớp chính xác, có thể không tìm được sách phù hợp. Nhưng AI có thể hiểu ý nghĩa gần đúng, ví dụ sách về architecture, distributed systems, backend engineering.

AI trong Library74 giúp:

- Tìm kiếm theo ngữ nghĩa, không chỉ theo keyword.
- Gợi ý sách cá nhân hóa theo hành vi người dùng.
- Đọc PDF sách để tạo tóm tắt, đối tượng đọc giả, tag.
- Biến nội dung sách thành vector để tìm kiếm thông minh.

1.2. Định Nghĩa Ngắn Gọn

AI service là một service Python chạy riêng, hỗ trợ backend Java trong các tác vụ cần xử lý ngôn ngữ, vector và recommendation.

Trong project Library74, AI nằm ở:

```
LMS_AI/
```

Luồng tổng quát:

```
Frontend -> Backend Java -> AI Service Python -> Database vector / LLM /  
model -> Backend -> Frontend
```

Trả lời hội đồng:

AI không thay thế backend, mà bổ sung khả năng hiểu nội dung và hành vi.
Backend vẫn giữ nghiệp vụ và bảo mật, còn AI service xử lý vector, semantic search, recommendation và metadata từ PDF.

2. Vì Sao AI Tách Thành Service Riêng?

2.1. Vấn Đề Thực Tế

Backend chính của project dùng Java Spring Boot. Nhưng phần AI thường cần thư viện Python:

- `sentence-transformers`
- `torch`
- `scikit-learn`
- `implicit`
- `PyMuPDF`
- FastAPI
- Celery

Nếu cố nhét tất cả vào Java backend, hệ thống sẽ nặng và khó bảo trì.

2.2. Lý Do Tách Service

AI được tách riêng vì:

- Python phù hợp với thư viện AI/ML hơn.
- AI xử lý PDF có thể lâu, không nên chặn request backend.
- Model embedding nên được load một lần trong process AI.
- Có thể scale AI worker riêng nếu cần.
- Backend Java giữ nghiệp vụ, AI Python giữ xử lý thông minh.

Kiến trúc:

```
LMS_FE      -> giao diện
LMS_BE      -> nghiệp vụ, bảo mật, API chính
LMS_AI      -> semantic search, recommendation, PDF ETL
PostgreSQL -> dữ liệu chung + vector
RabbitMQ    -> hàng đợi xử lý PDF
```

Trả lời hội đồng:

```
Project tách AI thành microservice vì AI cần Python và xử lý nặng. Backend
Java chỉ gọi AI qua API hoặc queue, nhờ vậy hệ thống rõ trách nhiệm, dễ
deploy và dễ mở rộng.
```

3. Công Nghệ AI Của Project

Công nghệ	Vai trò
Python	Ngôn ngữ chính của AI service
FastAPI	Tạo API semantic search và recommendation
Celery	Worker xử lý PDF nền
RabbitMQ	Queue nhận job xử lý sách
PyMuPDF	Đọc text từ PDF
Regex/Text cleaning	Làm sạch text trước khi chunk
Sentence Transformers/SBERT	Tạo embedding vector
PostgreSQL + pgvector	Lưu vector và tìm kiếm cosine
Gemini	Tóm tắt sách, tạo audience và tag
ALS recommendation	Gợi ý sách từ hành vi người dùng
Docker	Đóng gói AI service để deploy
Pytest	Kiểm thử AI

4. Ba Chức Năng AI Chính

AI service có 3 nhóm chức năng lớn:

Chức năng	Mục tiêu	Chạy khi nào
PDF ETL	Đọc PDF, chunk, vector hóa, tóm tắt, tag	Khi sách có PDF cần xử lý
Semantic Search	Tìm sách theo ý nghĩa câu hỏi	Khi người dùng search thông minh

Chức năng	Mục tiêu	Chạy khi nào
Recommendation	Gợi ý sách cá nhân hóa	Khi người dùng mở trang gợi ý/trang chủ

Nói ngắn:

```
ETL chuẩn bị dữ liệu AI.  
Semantic search dùng dữ liệu vector để tìm sách.  
Recommendation dùng hành vi người dùng để gợi ý sách.
```

5. ETL Là Gì?

5.1. Định Nghĩa

ETL là viết tắt của:

```
Extract -> Transform -> Load
```

Ý nghĩa:

- Extract: lấy dữ liệu thô.
- Transform: làm sạch, chia nhỏ, chuyển đổi.
- Load: lưu dữ liệu đã xử lý vào database.

Trong Library74:

```
PDF sách -> text -> chunks -> vectors -> database -> summary/tags/audience
```

5.2. Vì Sao Cần ETL PDF?

PDF là file dài. AI không thể tìm kiếm nhanh trực tiếp trên toàn bộ file mỗi lần user search.

Vì vậy hệ thống xử lý PDF trước:

1. Đọc text từ PDF.
2. Chia thành nhiều đoạn nhỏ.
3. Biến từng đoạn thành vector.
4. Lưu vector vào database.
5. Khi user search, chỉ cần so vector query với vector đã lưu.

Trả lời hội đồng:

ETL giúp chuyển PDF từ dữ liệu thô thành dữ liệu có thể tìm kiếm bằng AI. Hệ thống xử lý sách trước một lần, lưu vector vào database, nên lúc người dùng search thì phản hồi nhanh hơn.

6. Luồng Xử Lý PDF Trong Project

Khi thủ thư upload PDF cho một sách:

1. Backend có thông tin publication và PDF URL
2. Backend gửi message vào RabbitMQ
3. AI worker nhận job
4. Worker tải PDF
5. Worker đọc text bằng PyMuPDF
6. Làm sạch text
7. Chia text thành chunks
8. Tạo embedding vector cho từng chunk
9. Lưu vector vào database
10. Chọn một số chunk đại diện
11. Gọi Gemini để tạo summary, audience, tags
12. Update database
13. Callback về backend báo SUCCESS hoặc FAILED

Message gửi vào queue có dạng:

```
{
  "book_id": 101,
  "pdf_url": "https://example.com/books/book.pdf",
  "webhook_url": "http://backend:8080/api/ai/callback"
}
```

Trả lời hội đồng:

Xử lý PDF là tác vụ nặng và có thể lâu, nên backend không xử lý trực tiếp trong request. Backend đẩy job vào RabbitMQ, AI worker xử lý nền, sau đó callback về backend khi hoàn tất.

7. PyMuPDF Và Text Cleaning

7.1. PyMuPDF Là Gì?

PyMuPDF là thư viện Python dùng để đọc PDF và trích xuất text.

Trong project, phần này nằm ở:

```
LMS_AI/ai_etl/pdf_processing.py
```

7.2. Vì Sao Cần Làm Sạch Text?

Text lấy từ PDF thường không sạch:

- Header/footer lặp lại.
- Số trang.
- Ký tự lỗi.
- Xuống dòng sai.
- Khoảng trắng thừa.
- Từ bị tách dòng.

Nếu đưa text bẩn vào model, vector và summary sẽ kém chất lượng.

Trả lời hội đồng:

AI phụ thuộc rất nhiều vào chất lượng dữ liệu đầu vào. Vì text PDF thường nhiều, project làm sạch trước khi chunk và embedding để kết quả tìm kiếm/tóm tắt ổn định hơn.

8. Chunking Là Gì?

8.1. Vấn Đề Thực Tế

Một cuốn sách có thể dài hàng trăm trang. Không thể biến cả cuốn sách thành một vector duy nhất vì:

- Mất chi tiết.
- Query nhỏ khó khớp đúng đoạn.
- Model có giới hạn độ dài.

Vì vậy ta chia sách thành các đoạn nhỏ gọi là **chunk**.

8.2. Chunk Trong Project

Project dùng sliding window theo ký tự:

```
chunk_size = 1500  
overlap = 150
```

Ý nghĩa:

- Mỗi chunk khoảng 1500 ký tự.

- Hai chunk kế tiếp chồng lên nhau 150 ký tự để không mất ngữ cảnh ở điểm cắt.

Ví dụ:

```
Chunk 1: ký tự 0 -> 1500
Chunk 2: ký tự 1350 -> 2850
Chunk 3: ký tự 2700 -> 4200
```

Trả lời hội đồng:

Chunking giúp chia sách dài thành nhiều đoạn nhỏ để model hiểu tốt hơn. Overlap giúp giữ ngữ cảnh ở ranh giới giữa hai đoạn, tránh mất ý khi câu bị cắt.

9. Embedding Là Gì?

9.1. Định Nghĩa Dễ Hiểu

Embedding là cách biến văn bản thành một dãy số để máy tính so sánh ý nghĩa.

Ví dụ:

```
"lập trình backend" -> [0.12, -0.04, 0.88, ...]
"xây dựng API server" -> [0.10, -0.02, 0.83, ...]
```

Hai câu khác chữ nhưng gần nghĩa thì vector sẽ gần nhau.

9.2. Embedding Trong Project

Project dùng SBERT/Sentence Transformers để tạo vector 768 chiều.

File liên quan:

```
LMS_AI/ai_etl/embedding_client.py
LMS_AI/api_service.py
```

Vector được lưu vào database bằng **pgvector**.

Trả lời hội đồng:

Embedding giúp hệ thống so sánh ý nghĩa thay vì chỉ so khớp chữ. Nhờ vậy người dùng có thể tìm theo mô tả hoặc nhu cầu, không cần nhớ đúng tên sách.

10. pgvector Là Gì?

10.1. Vấn Đề Thực Tế

Sau khi tạo vector cho hàng nghìn đoạn văn, ta cần lưu và tìm vector gần nhất.

Database thường chỉ giỏi query số, chữ, ngày tháng. **pgvector** giúp PostgreSQL lưu và tìm vector.

10.2. Định Nghĩa

pgvector là extension của PostgreSQL để lưu vector và tìm kiếm similarity.

Ví dụ bảng vector:

```
ai_engine.publication_vectors
```

Mỗi dòng thường có:

```
publication_id  
chunk_text  
embedding vector(768)
```

Khi user search:

```
query text -> query vector -> tìm chunk vector gần nhất -> gom về  
publication_id
```

Trả lời hội đồng:

```
pgvector cho phép lưu embedding ngay trong PostgreSQL và query vector gần  
nhất. Nhờ vậy hệ thống không cần thêm một vector database riêng, giảm độ  
phức tạp triển khai.
```

11. Semantic Search Là Gì?

11.1. Keyword Search Và Semantic Search

Keyword search:

```
Tìm sách có chữ "backend"
```

Semantic search:

```
Tìm sách có ý nghĩa liên quan đến "xây dựng API, server, kiến trúc hệ thống"
```

Semantic search hiểu ý nghĩa gần đúng.

11.2. Luồng Semantic Search Trong Library74

Endpoint AI:

```
POST /api/v1/semantic-search
```

Input:

```
{  
  "query_text": "sách về thiết kế backend",  
  "limit": 10  
}
```

Luồng:

1. Backend nhận request search từ frontend
2. Backend gọi AI service
3. AI service embed query thành vector
4. AI service tìm chunk gần nhất trong pgvector
5. AI service gom kết quả về publication_ids
6. Backend lấy thông tin sách đầy đủ
7. Frontend hiển thị danh sách sách

Output AI service:

```
{  
  "publication_ids": [1, 5, 9]  
}
```

Trả lời hội đồng:

Semantic search không trả trực tiếp toàn bộ sách. AI service trả danh sách ID sách phù hợp, backend sau đó lấy dữ liệu nghiệp vụ đầy đủ để đảm bảo quyền, trạng thái và format response thống nhất.

12. Gemini Dừng Để Làm Gì?

12.1. Gemini Không Phải Phần Tìm Kiếm Chính

Trong project, Gemini không chạy mỗi khi người dùng search. Nếu làm vậy sẽ chậm và tốn quota.

Gemini chủ yếu dùng trong giai đoạn REDUCE của ETL:

- Tạo `ai_summary`.
- Xác định `ai_target_audience`.
- Gợi ý 5 tags.

12.2. Vì Sao Chỉ Gọi Gemini Ít Lần?

LLM như Gemini:

- Có giới hạn quota.
- Có thể chậm.
- Có thể lỗi tạm thời.
- Tốn chi phí nếu gọi nhiều.

Project tối ưu theo hướng:

Không gọi LLM cho từng chunk.
Chỉ chọn tối đa khoảng 12 chunk đại diện.
Gọi Gemini tối đa 1 lần cho mỗi sách để tổng hợp metadata.

Trả lời hội đồng:

Project dùng Gemini tiết kiệm ở bước tổng hợp metadata, không dùng cho mỗi truy vấn search. Search realtime dùng embedding và pgvector nên nhanh hơn, ổn định hơn và ít phụ thuộc quota LLM.

13. MAP-REDUCE Trong ETL Là Gì?

13.1. Ý Tưởng

MAP-REDUCE là cách xử lý dữ liệu lớn theo 2 bước:

- MAP: xử lý từng phần nhỏ.

- REDUCE: tổng hợp kết quả.

Trong project:

MAP: từng chunk -> embedding vector -> lưu database
REDUCE: chọn chunk đại diện -> Gemini tạo summary/tags/audience

Điểm tối ưu của project:

MAP không gọi LLM, chỉ tạo embedding.
REDUCE mới gọi LLM một lần.

Trả lời hội đồng:

Project áp dụng tư duy reduce-first để giảm chi phí LLM. Từng chunk chỉ cần embedding để search, còn summary/tags được tổng hợp một lần từ các chunk đại diện.

14. Fallback Là Gì?

14.1. Vấn Đề Thực Tế

LLM có thể lỗi:

- Quá tải.
- Timeout.
- Trả JSON sai format.
- Bị quota limit.

Nếu LLM lỗi mà cả pipeline fail, sách sẽ không được index.

14.2. Fallback Trong Project

Nếu Gemini fail, pipeline có thể dùng fallback deterministic:

- Vẫn lưu vector.
- Vẫn có metadata cơ bản.
- Hệ thống vẫn tìm kiếm được sách.

Trả lời hội đồng:

Fallback giúp hệ thống bền hơn. Nếu Gemini tạm thời lỗi, AI pipeline vẫn có thể index vector để search, thay vì bỏ toàn bộ quá trình xử lý sách.

15. Hash Skip Và Idempotency

15.1. Vấn Đề Thực Tế

Nếu cùng một PDF bị xử lý lại nhiều lần, hệ thống sẽ:

- Tốn thời gian.
- Tốn quota.
- Tạo vector trùng.
- Gây nhiễu dữ liệu.

15.2. Hash File

Project tính hash của file PDF.

Nếu file không đổi:

```
hash mới == hash đã lưu -> skip ETL
```

Nếu file đổi:

```
hash mới khác hash cũ -> xóa vector/tag cũ -> re-index sạch
```

15.3. Idempotency Là Gì?

Idempotency nghĩa là chạy lại cùng một thao tác nhiều lần nhưng kết quả cuối vẫn nhất quán.

Trả lời hội đồng:

```
AI ETL có idempotency bằng file hash. Nếu PDF không đổi thì bỏ qua xử lý lại, nếu PDF đổi thì re-index sạch. Cách này tiết kiệm tài nguyên và tránh dữ liệu trùng.
```

16. Celery Và RabbitMQ

16.1. Vì Sao Cần Queue?

Xử lý PDF có thể mất vài giây đến vài phút. Nếu backend chờ ngay trong request upload, người dùng sẽ phải chờ lâu.

Queue giúp xử lý nền.

```
Backend gửi job -> RabbitMQ giữ job -> Celery worker lấy job -> xử lý PDF
```

16.2. RabbitMQ Là Gì?

RabbitMQ là message broker, giống hàng đợi công việc.

Ví dụ:

```
book.processing.queue
```

16.3. Celery Worker Là Gì?

Celery worker là process Python chạy nền, lắng nghe queue và xử lý task.

File chính:

```
LMS_AI/worker.py
```

Trả lời hội đồng:

RabbitMQ lưu job xử lý sách, Celery worker xử lý job nền. Cách này giúp upload sách không bị kẹt vì AI xử lý lâu, đồng thời dễ retry nếu lỗi tạm thời.

17. FastAPI Service

17.1. FastAPI Là Gì?

FastAPI là framework Python để tạo API nhanh và có type rõ ràng.

Trong project, file chính:

```
LMS_AI/api_service.py
```

Service chạy ở port:

```
8001
```

Health check:

```
curl http://localhost:8001/health
```

Kỳ vọng:

```
{"status": "ok"}
```

17.2. Endpoint Chính

Endpoint	Vai trò
POST /api/v1/semantic-search	Tìm sách bằng vector
POST /api/v1/recommendations	Gợi ý sách cho user
GET /health	Kiểm tra service sống

Trả lời hội đồng:

FastAPI cung cấp API nội bộ cho backend Java. Backend gọi FastAPI để lấy kết quả semantic search hoặc recommendation, còn FastAPI tập trung vào xử lý model/vector.

18. Recommendation Là Gì?

18.1. Vấn Đề Thực Tế

Không phải người dùng nào cũng biết mình muốn tìm sách gì. Hệ thống nên gợi ý sách dựa trên hành vi.

Ví dụ:

- User xem nhiều sách lập trình.
- User thêm sách AI vào wishlist.
- User đã mượn sách database.

Hệ thống có thể gợi ý sách cùng chủ đề hoặc sách người tương tự cũng quan tâm.

18.2. Dữ Liệu Hành Vi

Project dùng các hành vi:

```
WATCH = 1  
WISHLIST = 5  
BORROWED = 10
```

Ý nghĩa:

- Xem sách thể hiện quan tâm nhẹ.
- Wishlist thể hiện quan tâm mạnh hơn.
- Mượn sách thể hiện quan tâm rõ nhất.

Trả lời hội đồng:

Recommendation dựa trên hành vi người dùng. Mỗi hành vi có trọng số khác nhau, vì mượn sách thể hiện nhu cầu mạnh hơn xem sách, còn wishlist mạnh hơn chỉ xem.

19. ALS Là Gì?

19.1. Định Nghĩa Dễ Hiểu

ALS là thuật toán recommendation dùng ma trận user-item.

Nói đơn giản:

Nếu user A và user B có hành vi giống nhau, thì sách user A thích có thể được gợi ý cho user B.

Nó học từ dữ liệu tương tác:

user_id, publication_id, action_type

19.2. Vì Sao Dùng ALS?

ALS phù hợp với implicit feedback, tức là hành vi không phải rating trực tiếp:

- Xem.
- Wishlist.
- Mượn.

Người dùng không cần chấm điểm tất cả sách, hệ thống vẫn học được từ hành vi.

Trả lời hội đồng:

ALS phù hợp vì dữ liệu thư viện chủ yếu là implicit feedback như xem, wishlist và mượn sách. Thuật toán học quan hệ user-item để gợi ý sách có khả năng phù hợp.

20. Cold Start Là Gì?

20.1. Vấn Đề Thực Tế

User mới chưa có lịch sử xem/mượn sách. Vậy gợi ý gì?

Đây gọi là **cold start**.

20.2. Fallback Gợi Ý

Project có các cách fallback:

- Nếu user chưa có tương tác: trả trending.
- Nếu user có ít tương tác nhưng chưa đủ profile ALS: dùng content-based theo vector sách đã xem.
- Chỉ gợi ý sách còn item **AVAILABLE**.

Trả lời hội đồng:

Cold start là trường hợp user mới chưa có dữ liệu hành vi. Project xử lý bằng trending hoặc content-based fallback để vẫn có gợi ý hợp lý thay vì trả rỗng.

21. Content-Based Recommendation

21.1. Định Nghĩa

Content-based recommendation gợi ý theo nội dung sách.

Ví dụ:

User xem sách Machine Learning
→ gợi ý sách có vector nội dung gần với Machine Learning

Khác với ALS:

- ALS dựa trên hành vi nhiều người.
- Content-based dựa trên nội dung sách.

Trả lời hội đồng:

Content-based giúp gợi ý khi dữ liệu hành vi chưa đủ. Nó dùng vector nội dung sách để tìm sách tương tự với sách user đã quan tâm.

22. AI Service Và Backend Java Phối Hợp Như Thế Nào?

Có 2 kiểu phối hợp:

22.1. Gọi Đồng Bộ Qua API

Dùng cho semantic search và recommendation.

```
Frontend -> Backend -> AI FastAPI -> Backend -> Frontend
```

Backend chờ AI trả kết quả rồi response cho frontend.

22.2. Xử Lý Bất Đồng Bộ Qua Queue

Dùng cho PDF ETL.

```
Backend -> RabbitMQ -> AI Worker -> callback Backend
```

Backend không chờ worker xử lý xong ngay.

Trả lời hội đồng:

Search/recommendation cần trả kết quả ngay nên backend gọi AI API đồng bộ.
Xử lý PDF lâu hơn nên dùng queue bất đồng bộ, worker xử lý nền rồi callback về backend.

23. Callback Là Gì?

23.1. Định Nghĩa

Callback là khi service A gọi lại service B sau khi xử lý xong.

Trong project:

```
Backend giao việc cho AI worker  
AI worker xử lý xong  
AI worker POST callback về backend
```

Callback thành công có thể có dạng:

```
{  
  "book_id": 101,  
  "status": "SUCCESS"  
}
```

Callback lỗi:

```
{
  "book_id": 101,
  "status": "FAILED",
  "error": "..."}
}
```

Trả lời hội đồng:

Callback giúp backend biết job AI đã hoàn tất hay thất bại. Vì PDF ETL chạy nền, backend không thể chờ trực tiếp trong request, nên callback là cách cập nhật trạng thái sau.

24. Database AI Lưu Gì?

24.1. Nhóm Dữ Liệu Chính

Dữ liệu	Mục đích
<code>publication_vectors</code>	Lưu chunk text và embedding
<code>publication_etl_runs</code>	Lưu file hash để skip xử lý lại
<code>publications.ai_summary</code>	Tóm tắt sách do AI tạo
<code>publications.ai_target_audience</code>	Độc giả mục tiêu
<code>publication_tags</code>	Liên kết sách với tag
<code>user_interactions</code>	Hành vi để recommendation

24.2. Vì Sao Vừa Lưu Vector Vừa Lưu Summary?

Vector dùng cho máy tính tìm kiếm.

Summary dùng cho người dùng đọc hiểu nhanh.

Trả lời hội đồng:

Vector phục vụ semantic search vì máy tính so sánh vector tốt hơn text thô. Summary và tags phục vụ trải nghiệm người dùng, giúp trang chi tiết sách dễ hiểu hơn.

25. Model Load Một Lần Là Gì?

25.1. Vấn Đề Thực Tế

Model AI khá nặng. Nếu mỗi request đều load model từ đầu:

- Rất chậm.
- Tốn RAM/CPU.
- Service không ổn định.

25.2. Cách Project Làm

FastAPI load SBERT một lần khi service start.

Celery worker cũng load model khi worker process init.

Trả lời hội đồng:

Model được load một lần khi service/worker khởi động để tránh tốn thời gian cho mỗi request. Sau đó các request dùng lại model đã load trong memory.

26. Connection Pool Là Gì?

26.1. Vấn Đề Thực Tế

Kết nối database tạo mới liên tục sẽ tốn thời gian.

Connection pool là một nhóm connection mở sẵn để dùng lại.

Trong project:

- FastAPI dùng pool cho request.
- Worker dùng pool khi ETL.
- Connection được mượn và trả lại.

Trả lời hội đồng:

Connection pool giúp giảm chi phí tạo kết nối database liên tục. AI service mượn connection khi cần query và trả lại pool sau khi xong.

27. Timeout Và Retry

27.1. Vì Sao Cần Timeout?

Nếu một query hoặc API call treo quá lâu, service có thể bị kẹt.

Project có:

- Timeout cho query nặng.

- Timeout khi gọi webhook.
- Timeout khi gọi LLM.

27.2. Vì Sao Cần Retry?

Một số lỗi chỉ là tạm thời:

- Network chậm chờn.
- Gemini quá tải.
- RabbitMQ/DB delay ngắn.

Retry giúp thử lại trước khi kết luận thất bại.

Trả lời hội đồng:

Timeout giúp hệ thống không bị treo vô hạn, retry giúp xử lý lỗi tạm thời.
Hai cơ chế này làm AI service bền hơn trong môi trường production.

28. Testing AI

28.1. Vì Sao AI Cũng Cần Test?

AI service không chỉ là model. Nó còn có:

- API contract.
- SQL query.
- PDF processing.
- JSON parsing từ Gemini.
- Worker message format.
- Recommendation logic.

Nếu không test, backend có thể gọi AI nhưng nhận format sai.

28.2. Test Trong Project

Các file test:

```
tests/test_pdf_processing.py
tests/test_llm_client.py
tests/test_pipeline_core.py
tests/test_api_service_contract.py
tests/test_recommender_engine.py
tests/test_worker_contract.py
tests/integration/test_api_contract_and_latency.py
```

Chạy test:

```
cd LMS_AI
pytest -q
```

Trả lời hội đồng:

AI service vẫn cần test vì phần quan trọng không chỉ là độ thông minh của model, mà còn là dữ liệu vào/ra đúng format, API đúng contract và pipeline xử lý không làm hỏng dữ liệu.

29. Docker Và Deploy AI

29.1. AI Chạy Ở Production Như Thế Nào?

Trong production, AI chạy bằng Docker Compose cùng stack:

```
lms_ai_api
lms_ai_worker
lms_rabbitmq
lms_postgres
lms_backend
```

AI API health:

```
docker exec lms_ai_api curl -fsS http://localhost:8001/health
```

Kỳ vọng:

```
{"status": "ok"}
```

29.2. Vì Sao Không Dùng localhost Trong Docker?

Trong Docker, `localhost` bên trong container là chính container đó, không phải máy host hay container khác.

Vì vậy production dùng tên service:

```
backend:8080
postgres:5432
```

```
rabbitmq:5672  
ai-api:8001
```

Trả lời hội đồng:

Trong Docker network, các container gọi nhau bằng tên service, không dùng localhost. Ví dụ AI gọi callback về backend bằng backend:8080 trong network nội bộ.

30. Một Luồng AI Thực Tế Đi Qua Hệ Thống

Ví dụ: thủ thư upload PDF sách mới.

1. Thủ thư tạo publication ở frontend
2. Frontend upload PDF hoặc lưu document URL
3. Backend lưu URL vào publication
4. Backend gửi job vào RabbitMQ
5. AI worker nhận job
6. Worker tải PDF
7. Worker trích text bằng PyMuPDF
8. Worker clean text và chia chunk
9. Worker tạo embedding bằng SBERT
10. Worker lưu vectors vào pgvector
11. Worker gọi Gemini tạo summary/audience/tags
12. Worker update database
13. Worker callback backend SUCCESS
14. Backend có thể gửi notification
15. User search semantic sẽ thấy sách tốt hơn

Nếu hội đồng hỏi "AI chạy lúc người dùng search hay lúc upload sách?", trả lời:

Cả hai nhưng khác nhiệm vụ. Lúc upload sách, AI worker xử lý PDF và lưu vector trước. Lúc người dùng search, AI service chỉ embed query và tìm vector gần nhất nên nhanh hơn nhiều.

31. Những Câu Hội Đồng Hay Hỏi

Câu 1: AI trong hệ thống dùng để làm gì?

Trả lời:

AI dùng để tìm kiếm ngữ nghĩa, gợi ý sách cá nhân hóa và xử lý PDF để tạo summary, audience, tags và vector. Nó giúp người dùng tìm sách theo nhu cầu

chứ không chỉ theo keyword.

Câu 2: Vì sao không để backend Java làm AI luôn?

Trả lời:

AI dùng nhiều thư viện Python như sentence-transformers, PyMuPDF, Celery và các thư viện ML. Tách AI thành service riêng giúp backend Java tập trung nghiệp vụ, còn Python service tập trung model/vector, dễ mở rộng và deploy độc lập.

Câu 3: Embedding là gì?

Trả lời:

Embedding là vector số đại diện cho ý nghĩa của văn bản. Hai đoạn văn gần nghĩa sẽ có vector gần nhau. Nhờ embedding, hệ thống tìm sách theo ý nghĩa thay vì chỉ khớp từ khóa.

Câu 4: Semantic search khác keyword search thế nào?

Trả lời:

Keyword search tìm chữ trùng, còn semantic search tìm ý nghĩa gần nhau. Ví dụ người dùng tìm 'xây dựng API server', semantic search có thể tìm sách backend dù title không chứa đúng cụm đó.

Câu 5: pgvector dùng để làm gì?

Trả lời:

pgvector giúp PostgreSQL lưu embedding vector và tìm vector gần nhất bằng cosine distance. Nhờ vậy project không cần triển khai thêm vector database riêng.

Câu 6: Vì sao phải chia PDF thành chunk?

Trả lời:

PDF quá dài để xử lý một lần và một vector duy nhất sẽ mất chi tiết. Chia chunk giúp lưu ý nghĩa từng đoạn nhỏ, từ đó search chính xác hơn. Overlap

giữ ngữ cảnh giữa các chunk.

Câu 7: Vì sao dùng RabbitMQ/Celery cho PDF?

Trả lời:

Xử lý PDF có thể lâu, nên không nên bắt người dùng chờ trong request. Backend đưa job vào RabbitMQ, Celery worker xử lý nền và callback khi xong.

Câu 8: Gemini dùng ở đâu?

Trả lời:

Gemini dùng trong bước tổng hợp metadata sau khi đọc PDF: tạo summary, audience và tags. Project không gọi Gemini cho mỗi lần search để tránh chậm và tốn quota.

Câu 9: Recommendation dựa trên dữ liệu nào?

Trả lời:

Recommendation dựa trên hành vi người dùng như xem sách, thêm wishlist và mượn sách. Các hành vi có trọng số khác nhau, sau đó thuật toán ALS học quan hệ user-item để gợi ý.

Câu 10: Cold start là gì và xử lý thế nào?

Trả lời:

Cold start là khi user mới chưa có dữ liệu hành vi. Project fallback bằng trending hoặc content-based recommendation để vẫn có gợi ý thay vì trả rỗng.

32. Cách Tự Học Code AI Trong Project

Không nên mở tất cả file cùng lúc. Học theo luồng.

32.1. Luồng Xử Lý PDF

Đọc theo thứ tự:

```
worker.py
ai_etl/config.py
ai_etl/pdf_processing.py
ai_etl/embedding_client.py
ai_etl/llm_client.py
ai_etl/db.py
ai_etl/pipeline.py
scripts/verify_etl_db.py
```

Câu hỏi cần trả lời:

- Worker nhận message format gì?
- PDF được đọc ở đâu?
- Text được chunk như thế nào?
- Vector được lưu vào bảng nào?
- Gemini được gọi ở bước nào?
- Callback về backend ra sao?

32.2. Luồng Semantic Search

Đọc theo thứ tự:

```
api_service.py
ai_etl/embedding_client.py
ai_gateway/config.py
tests/test_api_service_contract.py
```

Câu hỏi cần trả lời:

- Query text được biến thành vector ở đâu?
- SQL tìm vector gần nhất như thế nào?
- Vì sao output chỉ là **publication_ids**?

32.3. Luồng Recommendation

Đọc theo thứ tự:

```
api_service.py
ai_gateway/recommender.py
tests/test_recommender_engine.py
```

Câu hỏi cần trả lời:

- Hành vi user được lấy từ đâu?
- WATCH/WISHLIST/BORROWED khác trọng số thế nào?
- ALS dùng khi nào?

- Trending/content-based fallback dùng khi nào?

32.4. Luồng LLM/Gemini

Đọc theo thứ tự:

```
ai_etl/llm_client.py
ai_etl/tag_quality.py
tests/test_llm_client.py
tests/test_ai_metadata_quality.py
```

Câu hỏi cần trả lời:

- Prompt yêu cầu Gemini trả format gì?
- Nếu JSON lỗi thì xử lý thế nào?
- Tag được kiểm soát chất lượng ra sao?

33. Bản Tóm Tắt Một Phút

Nếu phải giải thích AI Library74 trong một phút:

AI trong Library74 là một service Python tách riêng khỏi backend Java. Nó có hai phần chính: FastAPI để phục vụ semantic search/recommendation realtime, và Celery worker để xử lý PDF nền qua RabbitMQ.

Khi sách có PDF, worker tải file, trích text bằng PyMuPDF, làm sạch, chia chunk, tạo embedding bằng SBERT và lưu vector vào PostgreSQL pgvector. Sau đó hệ thống chọn một số chunk đại diện và gọi Gemini một lần để tạo summary, audience và tags. Kết quả được lưu vào database và callback về backend.

Khi người dùng tìm kiếm, AI service biến câu query thành vector và tìm các chunk gần nhất để trả về publication_ids. Khi gợi ý sách, service dùng hành vi như xem, wishlist và mượn sách với ALS, có fallback trending/content-based cho user mới. Cách này giúp hệ thống tìm và gợi ý sách thông minh hơn nhưng vẫn giữ backend Java làm nơi quản lý nghiệp vụ và bảo mật.

34. Điều Quan Trọng Cần Nhớ

Nếu chỉ nhớ 10 ý, hãy nhớ:

1. AI service nằm ở **LMS_AI/** và viết bằng Python.
2. AI tách khỏi backend Java để xử lý model/vector/PDF tốt hơn.
3. PDF ETL biến sách từ file thô thành text, chunks, vectors, summary và tags.
4. Chunking giúp xử lý sách dài thành nhiều đoạn nhỏ.
5. Embedding là vector số biểu diễn ý nghĩa văn bản.

6. pgvector lưu vector trong PostgreSQL và tìm vector gần nhất.
7. Semantic search tìm theo ý nghĩa, không chỉ theo keyword.
8. Gemini dùng để tóm tắt/tag/audience, không gọi cho mỗi lần search.
9. RabbitMQ + Celery giúp xử lý PDF bất đồng bộ.
10. Recommendation dùng hành vi WATCH/WISHLIST/BORROWED và có fallback cho cold start.